



How Backend and DB Connect in Docker Compose

Great question! Let's break it down simply:

1. Shared Docker Network

When you run `docker-compose up`, Docker Compose automatically creates a dedicated network for your services. Every service (like your backend and database) joins this network and can communicate via service names as hostnames.^[1] ^[2]

2. Service Names as Hostnames

For example, if you define your database service as `mysql-db`, your backend can connect to the database using `host='mysql-db'`—not `localhost`. Docker's internal DNS resolves `mysql-db` to the actual database container IP.^[3] ^[2]

3. How to Define This in `compose.yml`

When both your backend and DB are listed under `services`: in your `docker-compose.yml`, they share this network by default:

```
services:
  mysql-db:
    image: mysql:8.0
    environment:
      MYSQL_USER: vehicledbuser
      MYSQL_PASSWORD: Mycloud@25
      MYSQL_DATABASE: vehicle_dashboard
      MYSQL_ROOT_PASSWORD: Mycloud@25
    ports:
      - "3306:3306"
  backend-subscriber:
    build: ./backend
    environment:
      DB_HOST: mysql-db      # This matches the service name!
      DB_USER: vehicledbuser
      DB_PASSWORD: Mycloud@25
      DB_NAME: vehicle_dashboard
    depends_on:
      - mysql-db
    ports:
      - "5000:5000"
```

Make sure your backend reads MySQL host from environment or sets it to `mysql-db`.

4. Why Not localhost?

localhost inside the backend container means “the backend container itself.” The DB isn’t running inside that container, so you need to use the Docker Compose service name instead.^[2]^[4] ^[3]

5. Connection Sequence

- Docker Compose launches both containers on the same network.
- Backend uses `host='mysql-db'` and connects over port 3306.
- Database service is reachable because it's on the same Docker network.

Summary:

Your backend and database containers communicate through the shared Docker network using service names (like `mysql-db`). No special config needed—this works right out of the box if both services are in `docker-compose.yml` and you use the service name in your backend's config.

Would you like to see how to update your Flask backend to use environment variables for the DB connection, or dive deeper into Compose networks?

✱

1. <https://www.warp.dev/terminus/docker-compose-networks>
2. <https://docs.docker.com/compose/how-tos/networking/>
3. <https://docs.docker.com/reference/compose-file/services/>
4. <https://stackoverflow.com/questions/47648792/communicating-between-different-docker-services-in-docker-compose>
5. <https://docs.docker.com/compose/gettingstarted/>
6. <https://learn.microsoft.com/en-us/azure/devops/pipelines/tasks/reference/docker-compose-v0?view=azure-pipelines>
7. <https://docs.divio.com/reference/docker-docker-compose/>
8. <https://docs.spring.io/spring-boot/reference/features/dev-services.html>
9. <https://learn.microsoft.com/en-us/dotnet/architecture/microservices/multi-container-microservice-net-applications/multi-container-applications-docker-compose>
10. <https://docs.docker.com/compose/intro/compose-application-model/>